Fonic HiFi Implementation Plan

This implementation plan provides a detailed, step-by-step guide for developing the Fonic HiFi iOS application. Each step is designed to build incrementally on the previous one while maintaining a functioning state throughout development.

Project Setup and Foundation

Step 1: Project Creation and Base Architecture

Detailed technical explanation

We'll create the Xcode project with the proper configuration for iOS 18+, Swift 6, and SwiftUI 5. We'll establish the foundational architecture following MVVM pattern with feature-based modules and set up the core folder structure that will house all components.

Task Breakdown

Create Xcode Project

- Create new iOS app using SwiftUI lifecycle
- Target iOS 18+, Swift 6, SwiftUI 5
- Configure app name, bundle identifier, and team
- /FonicHiFi.xcodeproj
- Create

Set Up Core Directory Structure

- Implement feature-based modular folder structure
- /FonicHiFi/App
- Create
- /FonicHiFi/Presentation
- Create
- /FonicHiFi/Domain
- Create
- /FonicHiFi/Data
- Create
- /FonicHiFi/Core
- Create

- /FonicHiFi/Utils
- Create
- /FonicHiFi/Resources
- Create
- /FonicHiFi/Tests
- Create

Configure Build Settings

- Enable strict concurrency checking
- Set Swift language version to Swift 6
- Enable recommended warnings
- /FonicHiFi.xcodeproj/project.pbxproj
- Update

Create App Entry Point

- Implement main app struct with environment setup
- /FonicHiFi/App/FonicHiFiApp.swift
- Create
- /FonicHiFi/App/AppDelegate.swift
- Create
- /FonicHiFi/App/SceneDelegate.swift
- Create
- /FonicHiFi/App/AppConfiguration.swift
- Create

Other Notes

- Ensure Xcode 16+ is installed
- Create a .gitignore file appropriate for Swift/iOS projects
- Consider setting up SwiftLint for code style enforcement

Step 2: Core Utilities and Extensions

Detailed technical explanation

We'll implement essential utilities and extensions that will be used throughout the app, including logging, error handling, and common Swift extensions. These will provide the foundation for consistent coding patterns across the application.

Task Breakdown

Implement Logging System

- Create a structured logging utility using os_log/Logger
- /FonicHiFi/Utils/Helpers/Logger.swift
- Create

Create Error Handling Framework

- Implement typed error system with user-friendly messages
- /FonicHiFi/Utils/Helpers/ErrorHandler.swift
- Create

Add Foundation Extensions

- Implement extensions for common Foundation types
- /FonicHiFi/Utils/Extensions/Foundation+Extensions.swift
- Create

Add SwiftUI Extensions

- Implement extensions for SwiftUI components
- /FonicHiFi/Utils/Extensions/SwiftUI+Extensions.swift
- Create

Create Constants

- Define app-wide constants
- /FonicHiFi/Utils/Constants/AppConstants.swift
- Create
- /FonicHiFi/Utils/Constants/UIConstants.swift
- Create

Other Notes

- Ensure extensions follow Swift API Design Guidelines
- Document all utilities and extensions with proper comments

Step 3: UI Theme and Design System

Detailed technical explanation

We'll establish the design system for the app, including typography, colors, spacing, and component styles. This will ensure a consistent visual language throughout the app and support dark mode, accessibility, and the minimalist hi-fi aesthetic.

Task Breakdown

Create Color System

- Define color palette with dark mode support
- /FonicHiFi/Presentation/UIState/ThemeManager.swift
- Create
- /FonicHiFi/Resources/Assets.xcassets/Colors
- Create

Implement Typography System

- Define text styles with dynamic type support
- /FonicHiFi/Presentation/UIState/Typography.swift
- Create

Create Component Styles

- Implement common UI component styles as ViewModifiers
- /FonicHiFi/Presentation/Common/ViewModifiers
- Create

Set Up Spacing System

- Define spacing constants and modifiers
- /FonicHiFi/Presentation/UIState/Spacing.swift
- Create

Implement Icon System

- Add SF Symbols and custom icons
- /FonicHiFi/Resources/Assets.xcassets/Icons
- Create

Other Notes

- Ensure all colors meet WCAG 2.2 AA contrast requirements
- Test typography with various dynamic type sizes

Step 4: Navigation and App State

Detailed technical explanation

We'll implement the navigation system and global app state management. This will provide the foundation for moving between different screens and maintaining consistent state across the app.

Task Breakdown

Create App State

- Implement global app state container
- /FonicHiFi/Presentation/UIState/AppState.swift
- Create

Implement Navigation System

- Create navigation coordinator using NavigationStack
- /FonicHiFi/Presentation/UIState/NavigationState.swift
- Create

Define Screen Routes

- Create enum-based route definitions
- /FonicHiFi/Presentation/UIState/Routes.swift
- Create

Set Up Main Navigation View

- Implement main navigation container
- /FonicHiFi/Presentation/Views/Common/MainNavigationView.swift
- Create

Create Tab Bar Navigation

- Implement tab-based navigation for main sections
- /FonicHiFi/Presentation/Views/Common/MainTabView.swift
- Create

Other Notes

- Ensure deep linking support is considered in the navigation architecture
- Test navigation with VoiceOver enabled

Core Services Implementation

Step 5: Domain Models and Interfaces

Detailed technical explanation

We'll define the core domain models and repository interfaces that represent the business logic of the application. These will serve as the contract between the data layer and the domain layer.

Task Breakdown

Create Audio Models

- Define Track, Album, Artist domain models
- /FonicHiFi/Domain/Models/Track.swift
- Create
- /FonicHiFi/Domain/Models/Album.swift
- Create
- /FonicHiFi/Domain/Models/Artist.swift
- Create
- /FonicHiFi/Domain/Models/AudioFormat.swift
- Create

Create Playlist Models

- Define Playlist and SmartPlaylist models
- /FonicHiFi/Domain/Models/Playlist.swift
- Create
- /FonicHiFi/Domain/Models/SmartPlaylist.swift
- Create

Define Repository Interfaces

- Create interfaces for data access
- /FonicHiFi/Domain/Interfaces/ILibraryRepository.swift
- Create
- /FonicHiFi/Domain/Interfaces/IPlaybackRepository.swift
- Create
- /FonicHiFi/Domain/Interfaces/IPlaylistRepository.swift
- Create
- /FonicHiFi/Domain/Interfaces/IMetadataRepository.swift
- Create

Implement Use Case Interfaces

- Define use case protocols for business logic
- /FonicHiFi/Domain/UseCases/Library/ImportMusicUseCase.swift
- Create
- /FonicHiFi/Domain/UseCases/Playback/PlayTrackUseCase.swift
- Create
- /FonicHiFi/Domain/UseCases/Playlists/CreatePlaylistUseCase.swift
- Create
- /FonicHiFi/Domain/UseCases/Metadata/EditMetadataUseCase.swift

- Create

Other Notes

- Ensure models are immutable where appropriate
- Use value types (structs) for domain models when possible

Step 6: Database and Storage Setup

Detailed technical explanation

We'll implement the data layer using SwiftData for local storage, with appropriate entity models, repositories, and data sources. This will provide the foundation for storing and retrieving music library data.

Task Breakdown

Create SwiftData Models

- Define entity models for SwiftData
- /FonicHiFi/Data/DTOs/TrackDTO.swift
- Create
- /FonicHiFi/Data/DTOs/AlbumDTO.swift
- Create
- /FonicHiFi/Data/DTOs/ArtistDTO.swift
- Create
- /FonicHiFi/Data/DTOs/PlaylistDTO.swift
- Create

Implement Data Mappers

- Create mappers between domain models and DTOs
- /FonicHiFi/Data/Mappers/TrackMapper.swift
- Create
- /FonicHiFi/Data/Mappers/AlbumMapper.swift
- Create
- /FonicHiFi/Data/Mappers/ArtistMapper.swift
- Create
- /FonicHiFi/Data/Mappers/PlaylistMapper.swift
- Create

Set Up SwiftData Source

- Implement SwiftData configuration and access
- /FonicHiFi/Data/DataSources/Local/SwiftDataSource.swift

- Create

Create File System Data Source

- Implement file system access for audio files
- /FonicHiFi/Data/DataSources/Local/FileSystemDataSource.swift
- Create

Implement User Defaults Data Source

- Set up storage for user preferences
- /FonicHiFi/Data/DataSources/Local/UserDefaultsDataSource.swift
- Create

Other Notes

- Ensure proper error handling for database operations
- Consider migration strategies for future schema changes

Step 7: Repository Implementations

Detailed technical explanation

We'll implement the repository interfaces defined in the domain layer, connecting them to the data sources. This will provide the concrete implementation for data access throughout the app.

Task Breakdown

Implement Library Repository

- Create library repository implementation
- /FonicHiFi/Data/Repositories/LibraryRepository.swift
- Create

Implement Playback Repository

- Create playback repository implementation
- /FonicHiFi/Data/Repositories/PlaybackRepository.swift
- Create

Implement Playlist Repository

- Create playlist repository implementation
- /FonicHiFi/Data/Repositories/PlaylistRepository.swift
- Create

Implement Metadata Repository

- Create metadata repository implementation
- /FonicHiFi/Data/Repositories/MetadataRepository.swift
- Create

Set Up Dependency Injection

- Create service provider for repository access
- /FonicHiFi/App/ServiceProvider.swift
- Create

Other Notes

- Ensure thread safety for repository implementations
- Use Swift Concurrency (async/await) for asynchronous operations

Step 8: Core Audio Services

Detailed technical explanation

We'll implement the core audio services that will be used by the playback engine, including audio session management, hardware detection, and format handling. These services will provide the foundation for high-quality audio playback.

Task Breakdown

Implement Audio Session Service

- Create service for managing iOS audio session
- /FonicHiFi/Core/Audio/AudioSession/AudioSessionService.swift
- Create

Create Hardware Detection Service

- Implement DAC detection and configuration
- /FonicHiFi/Core/Audio/AudioSession/HardwareDetectionService.swift
- Create

Implement Bit-Perfect Validator Service

- Create service to validate bit-perfect output
- /FonicHiFi/Core/Audio/Hardware/BitPerfectValidatorService.swift
- Create

Set Up Audio Format Services

- Implement services for handling various audio formats
- /FonicHiFi/Core/Audio/AudioEngine/AVAudioEngineAdapter.swift
- Create
- /FonicHiFi/Core/Audio/AudioEngine/SFBAudioEngineAdapter.swift
- Create
- /FonicHiFi/Core/Audio/AudioEngine/FFmpegAdapter.swift
- Create

Create Background Task Service

- Implement service for managing background tasks
- /FonicHiFi/Core/Background/BackgroundTaskService.swift
- Create

Other Notes

- Ensure proper handling of audio session interruptions
- Test with various audio hardware configurations if possible

Audio Playback Engine

Step 9: Audio Engine Implementation

Detailed technical explanation

We'll implement the core audio playback engine using AVAudioEngine as the primary backend, with support for SFBAudioEngine and FFmpegKit for extended format support. This will provide the foundation for high-quality audio playback.

Task Breakdown

Set Up Package Dependencies

- Add SFBAudioEngine and FFmpegKit dependencies
- /FonicHiFi.xcodeproj/project.pbxproj
- Update
- /Package.swift (if using Swift Package Manager)
- Create or Update

Implement Audio Engine Service

- Create main audio engine service
- /FonicHiFi/Core/Audio/AudioEngine/AudioEngineService.swift

- Create

Create Format Decoders

- Implement decoders for various audio formats
- /FonicHiFi/Core/Audio/AudioFormats/FLACDecoder.swift
- Create
- /FonicHiFi/Core/Audio/AudioFormats/ALACDecoder.swift
- Create
- /FonicHiFi/Core/Audio/AudioFormats/DSDDecoder.swift
- Create

Implement Engine Selection Logic

- Create logic for selecting appropriate audio engine
- /FonicHiFi/Core/Audio/AudioEngine/EngineSelector.swift
- Create

Set Up Performance Mode Manager

- Implement different performance modes
- /FonicHiFi/Core/Audio/AudioEngine/PerformanceModeManager.swift
- Create

Other Notes

- Test with various audio formats to ensure proper decoding
- Ensure proper memory management for audio buffers

Step 10: DSP and Audio Processing

Detailed technical explanation

We'll implement the DSP pipeline for audio processing, including equalizer, gain control, and other audio effects. This will be implemented in phases as outlined in the technical specification.

Task Breakdown

Implement Equalizer Service (Phase 1)

- Create 10-band equalizer using AVAudioUnitEQ
- /FonicHiFi/Core/Audio/AudioProcessing/EqualizerService.swift
- Create

Create Gain Control Service (Phase 1)

- Implement basic volume adjustment
- /FonicHiFi/Core/Audio/AudioProcessing/GainControlService.swift
- Create

Set Up Sample Rate Conversion (Phase 1)

- Implement sample rate conversion for hardware compatibility
- /FonicHiFi/Core/Audio/AudioProcessing/SampleRateConverter.swift
- Create

Create DSP Chain

- Implement audio processing pipeline
- /FonicHiFi/Core/Audio/AudioProcessing/DSPChain.swift
- Create

Implement Preset Management

- Create system for managing equalizer presets
- /FonicHiFi/Core/Audio/AudioProcessing/EqualizerPresets.swift
- Create

Other Notes

- Ensure DSP operations are efficient and don't cause audio glitches
- Test with various audio hardware to ensure compatibility

Step 11: Waveform Generation and Visualization

Detailed technical explanation

We'll implement the waveform generation and visualization system, with support for different performance modes and file formats. This will provide visual feedback during playback and browsing.

Task Breakdown

Create Waveform Generator Service

- Implement service for generating waveforms from audio files
- /FonicHiFi/Core/Audio/Analysis/WaveformGenerator.swift
- Create

Implement Waveform Compression

- Create delta encoding compression for waveform data
- /FonicHiFi/Core/Audio/Analysis/WaveformCompression.swift
- Create

Create Waveform View Model

- Implement view model for waveform visualization
- /FonicHiFi/Presentation/ViewModels/Player/WaveformViewModel.swift
- Create

Implement Waveform View

- Create SwiftUI view for waveform visualization
- /FonicHiFi/Presentation/Views/Player/WaveformView.swift
- Create

Set Up Adaptive Rendering

- Implement different rendering modes based on performance settings
- /FonicHiFi/Presentation/Views/Player/Components/AdaptiveWaveformRenderer.swift
- Create

Other Notes

- Ensure waveform generation happens in background threads
- Test with various audio formats and file sizes

Step 12: Playback Controls and Queue Management

Detailed technical explanation

We'll implement the playback controls and queue management system, including play, pause, skip, and queue manipulation. This will provide the user interface for controlling audio playback.

Task Breakdown

Implement Player View Model

- Create view model for player controls
- /FonicHiFi/Presentation/ViewModels/Player/PlayerViewModel.swift
- Create

Create Queue Management

- Implement playback queue with reordering support
- /FonicHiFi/Domain/UseCases/Playback/QueueManagementUseCase.swift
- Create

Implement Player View

- Create main player interface
- /FonicHiFi/Presentation/Views/Player/PlayerView.swift
- Create

Create Mini Player

- Implement persistent mini player for navigation
- /FonicHiFi/Presentation/Views/Player/MiniPlayerView.swift
- Create

Set Up Background Playback

- Configure app for background audio playback
- /FonicHiFi/Core/Audio/AudioSession/BackgroundPlaybackConfigurator.swift
- Create
- /FonicHiFi/Resources/Info.plist
- Update

Other Notes

- Ensure proper handling of audio interruptions
- Test background playback with various scenarios

Music Library Management

Step 13: File Import and Scanning

Detailed technical explanation

We'll implement the file import and scanning system, allowing users to import music from the Files app or external drives. This will provide the foundation for building the music library.

Task Breakdown

Create File Manager Service

- Implement service for file system operations
- /FonicHiFi/Core/FileSystem/FileManagerService.swift
- Create

Implement Import Service

- Create service for importing music files
- /FonicHiFi/Core/FileSystem/ImportService.swift
- Create

Set Up Storage Monitor

- Implement service for monitoring storage space
- /FonicHiFi/Core/FileSystem/StorageMonitorService.swift
- Create

Create Import Use Case

- Implement business logic for importing music
- /FonicHiFi/Domain/UseCases/Library/ImportMusicUseCase.swift
- Update

Implement Import UI

- Create user interface for importing files
- /FonicHiFi/Presentation/Views/Library/ImportView.swift
- Create
- /FonicHiFi/Presentation/ViewModels/Library/ImportViewModel.swift
- Create

Other Notes

- Ensure proper error handling for file access issues
- Test with various file sources and formats

Step 14: Metadata Parsing and Indexing

Detailed technical explanation

We'll implement the metadata parsing and indexing system, extracting and storing metadata from audio files. This will provide the foundation for organizing and browsing the music library.

Task Breakdown

Add TagLib Integration

- Set up TagLib for metadata parsing
- /FonicHiFi.xcodeproj/project.pbxproj
- Update

Create Tag Parser Service

- Implement service for parsing audio tags
- /FonicHiFi/Core/Metadata/TagParserService.swift
- Create

Implement Indexing Service

- Create service for indexing music library
- /FonicHiFi/Core/Background/IndexingService.swift
- Create

Set Up Library Scanning

- Implement logic for scanning and indexing library
- /FonicHiFi/Domain/UseCases/Library/ScanLibraryUseCase.swift
- Create

Create Progress UI

- Implement UI for showing scanning progress
- /FonicHiFi/Presentation/Views/Library/ScanningProgressView.swift
- Create
- /FonicHiFi/Presentation/ViewModels/Library/ScanningProgressViewModel.swift
- Create

Other Notes

- Ensure efficient processing of large libraries
- Implement proper error handling for corrupt or incomplete tags

Step 15: Library Browsing and Navigation

Detailed technical explanation

We'll implement the library browsing and navigation system, allowing users to browse their music by artists, albums, genres, and other criteria. This will provide the main interface for accessing the music library.

Task Breakdown

Create Library View Model

- Implement view model for library browsing
- /FonicHiFi/Presentation/ViewModels/Library/LibraryViewModel.swift
- Create

Implement Artist List View

- Create view for browsing artists
- /FonicHiFi/Presentation/Views/Library/ArtistListView.swift
- Create
- /FonicHiFi/Presentation/ViewModels/Library/ArtistListViewModel.swift
- Create

Implement Album List View

- Create view for browsing albums
- /FonicHiFi/Presentation/Views/Library/AlbumListView.swift
- Create
- /FonicHiFi/Presentation/ViewModels/Library/AlbumListViewModel.swift
- Create

Create Track List View

- Implement view for browsing tracks
- /FonicHiFi/Presentation/Views/Library/TrackListView.swift
- Create
- /FonicHiFi/Presentation/ViewModels/Library/TrackListViewModel.swift
- Create

Set Up Genre and Quality Filters

- Create filtering by genre and audio quality
- /FonicHiFi/Presentation/Views/Library/GenreListView.swift
- Create
- /FonicHiFi/Presentation/Views/Library/QualityFilterView.swift
- Create

Other Notes

- Ensure smooth scrolling with large libraries
- Implement proper loading states and error handling

Step 16: Search and Filtering

Detailed technical explanation

We'll implement the search and filtering system, allowing users to find music by various criteria. This will enhance the library browsing experience with powerful search capabilities.

Task Breakdown

Implement Search Service

- Create service for searching the music library
- /FonicHiFi/Domain/UseCases/Library/SearchLibraryUseCase.swift
- Create

Create Search View Model

- Implement view model for search functionality
- /FonicHiFi/Presentation/ViewModels/Library/SearchViewModel.swift
- Create

Implement Search View

- Create user interface for searching
- /FonicHiFi/Presentation/Views/Library/SearchView.swift
- Create

Set Up Filter Components

- Implement reusable filter components
- /FonicHiFi/Presentation/Views/Library/Components/FilterChip.swift
- Create
- /FonicHiFi/Presentation/Views/Library/Components/FilterSheet.swift
- Create

Create Search Results View

- Implement view for displaying search results
- /FonicHiFi/Presentation/Views/Library/SearchResultsView.swift
- Create

Other Notes

- Optimize search for performance with large libraries
- Implement proper highlighting of search terms

Smart Playlists and Metadata Editing

Step 17: Smart Playlist Engine

Detailed technical explanation

We'll implement the smart playlist engine, allowing users to create dynamic playlists based on various criteria. This will provide powerful organization capabilities for the music library.

Task Breakdown

Create Filter Rule Model

- Define model for filter rules
- /FonicHiFi/Domain/Models/FilterRule.swift
- Create

Implement Query Builder

- Create service for building database queries from filter rules
- /FonicHiFi/Data/Repositories/QueryBuilder.swift
- Create

Create Smart Playlist Service

- Implement service for managing smart playlists
- /FonicHiFi/Domain/UseCases/Playlists/SmartFilterUseCase.swift
- Create

Implement Smart Playlist Editor View Model

- Create view model for editing smart playlists
- /FonicHiFi/Presentation/ViewModels/Playlists/SmartPlaylistEditorViewModel.swift
- Create

Create Smart Playlist Editor View

- Implement user interface for creating and editing smart playlists
- /FonicHiFi/Presentation/Views/Playlists/SmartPlaylistEditorView.swift
- Create
- /FonicHiFi/Presentation/Views/Playlists/Components/FilterRuleView.swift
- Create

Other Notes

- Ensure efficient query execution for complex filters
- Implement proper validation for filter rules

Step 18: Playlist Management

Detailed technical explanation

We'll implement the playlist management system, allowing users to create, edit, and manage playlists. This will complement the smart playlist functionality with manual playlist capabilities.

Task Breakdown

Create Playlist Service

- Implement service for managing playlists
- /FonicHiFi/Domain/UseCases/Playlists/ManagePlaylistsUseCase.swift
- Create

Implement Playlists View Model

- Create view model for playlist management
- /FonicHiFi/Presentation/ViewModels/Playlists/PlaylistsViewModel.swift
- Create

Create Playlists View

- Implement user interface for browsing playlists
- /FonicHiFi/Presentation/Views/Playlists/PlaylistsView.swift
- Create

Implement Playlist Detail View

- Create view for displaying playlist contents
- /FonicHiFi/Presentation/Views/Playlists/PlaylistDetailView.swift
- Create
- /FonicHiFi/Presentation/ViewModels/Playlists/PlaylistDetailViewModel.swift
- Create

Set Up Playlist Creation UI

- Implement interface for creating new playlists
- /FonicHiFi/Presentation/Views/Playlists/CreatePlaylistView.swift
- Create

Other Notes

- Ensure proper handling of playlist reordering
- Implement efficient updates when tracks are added or removed

Step 19: Metadata Editor

Detailed technical explanation

We'll implement the metadata editor, allowing users to view and edit audio file metadata. This will provide powerful tools for organizing and maintaining the music library.

Task Breakdown

Create Metadata Editor Service

- Implement service for editing metadata
- /FonicHiFi/Core/Metadata/MetadataEditorService.swift
- Create

Implement Tag Writer Service

- Create service for writing tags to audio files
- /FonicHiFi/Core/Metadata/TagWriterService.swift
- Create

Create Metadata Editor View Model

- Implement view model for metadata editing
- /FonicHiFi/Presentation/ViewModels/Metadata/MetadataEditorViewModel.swift
- Create

Implement Metadata Editor View

- Create user interface for editing metadata
- /FonicHiFi/Presentation/Views/Metadata/MetadataEditorView.swift
- Create
- /FonicHiFi/Presentation/Views/Metadata/Components/MetadataFieldEditor.swift
- Create

Set Up Artwork Management

- Implement artwork editing capabilities
- /FonicHiFi/Presentation/Views/Metadata/ArtworkEditorView.swift
- Create

Other Notes

- Ensure non-destructive editing with backup capability
- Implement proper validation for metadata fields

Step 20: Batch Editing

Detailed technical explanation

We'll implement batch editing capabilities, allowing users to edit metadata for multiple files simultaneously. This will enhance the metadata editor with powerful bulk editing tools.

Task Breakdown

Create Batch Edit Models

- Define models for batch changes
- /FonicHiFi/Domain/Models/MetadataChanges.swift
- Create

Implement Batch Edit Use Case

- Create business logic for batch editing
- /FonicHiFi/Domain/UseCases/Metadata/BatchEditUseCase.swift
- Create

Create Batch Editor View Model

- Implement view model for batch editing
- /FonicHiFi/Presentation/ViewModels/Metadata/BatchEditorViewModel.swift
- Create

Implement Batch Editor View

- Create user interface for batch editing
- /FonicHiFi/Presentation/Views/Metadata/BatchEditorView.swift
- Create
- /FonicHiFi/Presentation/Views/Metadata/Components/BatchChangeEditor.swift
- Create

Set Up Preview System

- Implement preview of batch changes
- /FonicHiFi/Presentation/Views/Metadata/BatchPreviewView.swift
- Create

Other Notes

- Ensure efficient processing of large batches
- Implement proper progress tracking and cancellation

User Interface and Experience

Step 21: Player Interface

Detailed technical explanation

We'll implement the main player interface, providing a rich and intuitive experience for controlling playback. This will be the central point for interacting with the currently playing track.

Task Breakdown

Create Player UI Components

- Implement reusable player components
- /FonicHiFi/Presentation/Views/Player/Components/PlaybackControlsView.swift
- Create
- /FonicHiFi/Presentation/Views/Player/Components/TrackInfoView.swift
- Create
- /FonicHiFi/Presentation/Views/Player/Components/ProgressSlider.swift
- Create

Implement Full-Screen Player

- Create immersive player experience
- /FonicHiFi/Presentation/Views/Player/FullScreenPlayerView.swift
- Create

Set Up Queue View

- Implement interface for managing playback queue
- /FonicHiFi/Presentation/Views/Player/QueueView.swift
- Create
- /FonicHiFi/Presentation/ViewModels/Player/QueueViewModel.swift
- Create

Create Audio Quality Indicators

- Implement visual indicators for audio quality
- /FonicHiFi/Presentation/Views/Common/AudioQualityBadge.swift

- Create

Implement Lyrics Display

- Create interface for displaying synchronized lyrics
- /FonicHiFi/Presentation/Views/Player/LyricsView.swift
- Create

Other Notes

- Ensure smooth animations and transitions
- Test interface with various screen sizes

Step 22: Settings and Preferences

Detailed technical explanation

We'll implement the settings and preferences system, allowing users to customize the app's behavior. This will provide control over playback, appearance, and other aspects of the app.

Task Breakdown

Create Settings Models

- Define models for app settings
- /FonicHiFi/Domain/Models/AppSettings.swift
- Create
- /FonicHiFi/Domain/Models/AudioSettings.swift
- Create

Implement Settings Service

- Create service for managing settings
- /FonicHiFi/Data/Repositories/SettingsRepository.swift
- Create

Create Settings View Model

- Implement view model for settings interface
- /FonicHiFi/Presentation/ViewModels/Settings/SettingsViewModel.swift
- Create

Implement Settings Views

- Create user interface for settings

- /FonicHiFi/Presentation/Views/Settings/SettingsView.swift
- Create
- /FonicHiFi/Presentation/Views/Settings/AudioSettingsView.swift
- Create
- /FonicHiFi/Presentation/Views/Settings/AppearanceSettingsView.swift
- Create
- /FonicHiFi/Presentation/Views/Settings/LibrarySettingsView.swift
- Create

Set Up Performance Mode Settings

- Implement interface for selecting performance mode
- /FonicHiFi/Presentation/Views/Settings/PerformanceModeView.swift
- Create

Other Notes

- Ensure settings are persisted properly
- Implement immediate application of setting changes where appropriate

Step 23: Accessibility Implementation

Detailed technical explanation

We'll implement comprehensive accessibility features throughout the app, ensuring it's usable by everyone. This includes VoiceOver support, dynamic type, and other accessibility features.

Task Breakdown

Add VoiceOver Support

- Implement proper accessibility labels and hints
- Update all view files to include accessibility modifiers
- Update

Set Up Dynamic Type

- Ensure text scales properly with system settings
- Update typography system and text components
- Update

Implement Reduce Motion Support

- Create alternative animations for users with motion sensitivity
- /FonicHiFi/Presentation/UIState/AnimationSettings.swift

- Create

Add High Contrast Support

- Ensure sufficient contrast in all UI elements
- Update color system and UI components
- Update

Create Accessibility Settings

- Implement additional accessibility options
- /FonicHiFi/Presentation/Views/Settings/AccessibilitySettingsView.swift
- Create

Other Notes

- Test with VoiceOver and other accessibility features enabled
- Ensure all interactive elements are properly labeled

Step 24: Onboarding and Help

Detailed technical explanation

We'll implement the onboarding experience and help system, guiding users through the app's features. This will help users get the most out of the app and understand its capabilities.

Task Breakdown

Create Onboarding Flow

- Implement first-launch experience
- /FonicHiFi/Presentation/Views/Onboarding/OnboardingView.swift
- Create
- /FonicHiFi/Presentation/ViewModels/Onboarding/OnboardingViewModel.swift
- Create

Implement Feature Tours

- Create guided tours of key features
- /FonicHiFi/Presentation/Views/Onboarding/FeatureTourView.swift
- Create

Set Up Help System

- Implement contextual help throughout the app

- /FonicHiFi/Presentation/Views/Common/HelpOverlay.swift
- Create

Create Privacy Onboarding

- Implement privacy-focused onboarding screens
- /FonicHiFi/Presentation/Views/Onboarding/PrivacyOnboardingView.swift
- Create

Add Tooltips and Hints

- Implement subtle guidance for complex features
- /FonicHiFi/Presentation/Views/Common/TooltipView.swift
- Create

Other Notes

- Ensure onboarding can be skipped and revisited
- Test onboarding flow with new users if possible

Cloud Integration and Advanced Features

Step 25: Cloud Storage Integration

Detailed technical explanation

We'll implement integration with cloud storage services, allowing users to access their music from Google Drive, Dropbox, and other services. This will extend the app's capabilities beyond local files.

Task Breakdown

Set Up OAuth Authentication

- Implement secure authentication for cloud services
- /FonicHiFi/Core/Network/OAuthManager.swift
- Create

Create Cloud Data Sources

- Implement data sources for cloud services
- /FonicHiFi/Data/DataSources/Remote/CloudDataSource.swift
- Create
- /FonicHiFi/Data/DataSources/Remote/GoogleDriveDataSource.swift
- Create

- /FonicHiFi/Data/DataSources/Remote/DropboxDataSource.swift
- Create

Implement Cloud Repository

- Create repository for cloud file access
- /FonicHiFi/Data/Repositories/CloudRepository.swift
- Create

Create Cloud Browser

- Implement interface for browsing cloud files
- /FonicHiFi/Presentation/Views/Cloud/CloudBrowserView.swift
- Create
- /FonicHiFi/Presentation/ViewModels/Cloud/CloudBrowserViewModel.swift
- Create

Set Up Account Management

- Implement interface for managing cloud accounts
- /FonicHiFi/Presentation/Views/Settings/AccountSettingsView.swift
- Create

Other Notes

- Ensure secure storage of authentication tokens
- Implement proper error handling for network issues

Step 26: Download Management

Detailed technical explanation

We'll implement the download management system, allowing users to download music from cloud services for offline playback. This will provide seamless access to music regardless of connectivity.

Task Breakdown

Create Download Manager

- Implement service for managing downloads
- /FonicHiFi/Core/Network/DownloadManager.swift
- Create

Set Up Background Downloads

- Configure app for background download support
- /FonicHiFi/Core/Background/BackgroundDownloadService.swift
- Create

Implement Download Queue

- Create system for managing download priorities
- /FonicHiFi/Domain/UseCases/Downloads/DownloadQueueUseCase.swift
- Create

Create Download UI

- Implement interface for managing downloads
- /FonicHiFi/Presentation/Views/Downloads/DownloadsView.swift
- Create
- /FonicHiFi/Presentation/ViewModels/Downloads/DownloadsViewModel.swift
- Create

Set Up Offline Indicators

- Implement visual indicators for offline availability
- /FonicHiFi/Presentation/Views/Common/OfflineIndicator.swift
- Create

Other Notes

- Ensure proper handling of interrupted downloads
- Implement bandwidth-aware downloading

Step 27: Advanced Audio Features

Detailed technical explanation

We'll implement advanced audio features, including ReplayGain support, crossfade, and enhanced equalizer capabilities. These features will be part of Phase 2 of the DSP implementation.

Task Breakdown

Implement ReplayGain Support

- Create service for ReplayGain processing
- /FonicHiFi/Core/Audio/AudioProcessing/ReplayGain.swift
- Create

Add Crossfade Support

- Implement crossfade between tracks
- /FonicHiFi/Core/Audio/AudioProcessing/CrossfadeProcessor.swift
- Create

Enhance Equalizer

- Implement advanced equalizer with custom presets
- /FonicHiFi/Core/Audio/AudioProcessing/EqualizerService.swift
- Update

Create Audio Effects UI

- Implement interface for controlling audio effects
- /FonicHiFi/Presentation/Views/Player/AudioEffectsView.swift
- Create
- /FonicHiFi/Presentation/ViewModels/Player/AudioEffectsViewModel.swift
- Create

Set Up A-B Repeat

- Implement feature for repeating specific sections
- /FonicHiFi/Core/Audio/AudioEngine/ABRepeatService.swift
- Create

Other Notes

- Ensure effects don't introduce audio artifacts
- Test with various audio formats and hardware

Step 28: Security and Privacy Features

Detailed technical explanation

We'll implement comprehensive security and privacy features, ensuring user data is protected and privacy is maintained. This includes secure storage, privacy controls, and clear user communication.

Task Breakdown

Implement Secure Storage

- Create service for secure data storage
- /FonicHiFi/Core/Security/SecureStorageService.swift
- Create

Set Up Privacy Controls

- Implement granular privacy settings
- /FonicHiFi/Presentation/Views/Settings/PrivacySettingsView.swift
- Create

Create Privacy Indicators

- Implement visual indicators for privacy status
- /FonicHiFi/Presentation/Views/Common/PrivacyIndicator.swift
- Create

Add Network Security

- Implement certificate pinning and secure connections
- /FonicHiFi/Core/Network/NetworkSecurityConfigurator.swift
- Create

Create Data Management Tools

- Implement tools for managing user data
- /FonicHiFi/Presentation/Views/Settings/DataManagementView.swift
- Create

Other Notes

- Ensure compliance with privacy regulations
- Implement clear privacy notices and consent flows

Testing and Finalization

Step 29: Unit and Integration Testing

Detailed technical explanation

We'll implement comprehensive unit and integration tests to ensure the app functions correctly. This will provide confidence in the app's reliability and help catch regressions.

Task Breakdown

Create Domain Layer Tests

- Implement tests for domain models and use cases
- /FonicHiFi/Tests/UnitTests/Domain/
- Create

Add Data Layer Tests

- Implement tests for repositories and data sources
- /FonicHiFi/Tests/UnitTests/Data/
- Create

Create Core Services Tests

- Implement tests for core services
- /FonicHiFi/Tests/UnitTests/Core/
- Create

Add Integration Tests

- Implement tests for component interactions
- /FonicHiFi/Tests/IntegrationTests/
- Create

Create Test Utilities

- Implement helpers for testing
- /FonicHiFi/Tests/TestUtilities/
- Create

Other Notes

- Aim for high test coverage of critical components
- Implement proper mocking for external dependencies

Step 30: Performance Optimization

Detailed technical explanation

We'll optimize the app's performance, focusing on smooth playback, efficient library browsing, and battery life. This will ensure the app provides a great experience even with large libraries.

Task Breakdown

Implement Library Benchmarking

- Create tools for measuring library performance
- /FonicHiFi/Utils/Helpers/PerformanceBenchmark.swift
- Create

Optimize Memory Usage

- Implement efficient memory management
- Review and update all view models and services
- Update

Enhance Background Processing

- Optimize background tasks for efficiency
- /FonicHiFi/Core/Background/BackgroundTaskOptimizer.swift
- Create

Implement Battery Optimizations

- Create battery-saving measures
- /FonicHiFi/Core/Audio/AudioEngine/BatteryOptimizer.swift
- Create

Add Performance Logging

- Implement detailed performance monitoring
- /FonicHiFi/Utils/Helpers/PerformanceLogger.swift
- Create

Other Notes

- Test with large libraries (100,000+ tracks)
- Profile memory and CPU usage during intensive operations

Step 31: Final Polish and Localization

Detailed technical explanation

We'll add final polish to the app, including animations, transitions, and localization. This will ensure the app provides a delightful and accessible experience for all users.

Task Breakdown

Enhance Animations and Transitions

- Refine animations throughout the app
- /FonicHiFi/Presentation/UIState/AnimationConfigurator.swift
- Create

Implement Localization

- Set up localization infrastructure

- /FonicHiFi/Resources/Localizable.strings
- Create
- /FonicHiFi/Resources/Localizable.stringsdict
- Create

Add Final UI Polish

- Refine visual details throughout the app
- Review and update all view files
- Update

Create App Icon and Launch Screen

- Implement final app icon and launch experience
- /FonicHiFi/Resources/Assets.xcassets/AppIcon.appiconset
- Create
- /FonicHiFi/Resources/LaunchScreen.storyboard
- Create

Perform Final Accessibility Review

- Ensure comprehensive accessibility support
- Review all views for accessibility issues
- Update

Other Notes

- Test on various devices and screen sizes
- Ensure consistent visual language throughout the app

Step 32: Documentation and Deployment Preparation

Detailed technical explanation

We'll prepare the app for deployment, including documentation, release notes, and App Store assets. This will ensure a smooth release process and provide resources for future development.

Task Breakdown

Create Developer Documentation

- Document architecture and key components
- /Documentation/Architecture.md
- Create
- /Documentation/Components.md

- Create

Prepare Release Notes

- Document features and changes
- /Documentation/ReleaseNotes.md
- Create

Create App Store Assets

- Prepare screenshots and promotional materials
- /AppStoreAssets/
- Create

Configure App Store Connect

- Set up app listing and metadata
- External task - requires App Store Connect access

Prepare TestFlight Distribution

- Configure app for beta testing
- /FonicHiFi.xcodeproj/project.pbxproj
- Update

Other Notes

- Ensure all required App Store information is prepared
- Consider creating a beta testing plan

Implementation Notes

This implementation plan is designed to build the Fonic HiFi app incrementally, with each step building on the previous ones while maintaining a functioning state. The steps are organized to establish the foundation first, then add core services, followed by feature implementation, and finally polish and optimization.

Key dependencies are managed in the proper order: 1. Project setup and architecture (Steps 1-4) 2. Domain models and data layer (Steps 5-7) 3. Core audio services (Steps 8-12) 4. Library management (Steps 13-16) 5. Smart playlists and metadata editing (Steps 17-20) 6. User interface and experience (Steps 21-24) 7. Cloud integration and advanced features (Steps 25-28) 8. Testing and finalization (Steps 29-32)

Each step is designed to result in a buildable state of the app, with incremental functionality added at each stage. This allows for continuous testing and validation throughout the development process.

The plan addresses all aspects of the technical specification, including: - Modular architecture with MVVM pattern - High-quality audio playback with bit-perfect output - Comprehensive library management - Smart playlists and filtering - Metadata editing and batch operations - Cloud integration and offline playback - Privacy-first and offline-first design - Accessibility and localization

By following this plan, the development team will be able to implement the Fonic HiFi app in a structured and efficient manner, ensuring all requirements are met and the app provides a great experience for audiophile users.